Avance Proyecto 2:
Pipeline IoT/Edge

IL355 - Programación de Sistemas Avanzados

Backlog

Infraestructura

- Crear estructura de carpetas
- Configurar `.gitignore`
- Crear `docker-compose.yml`
- Crear `README.md` inicial
- Documentar red (VPN)

Sistema en Rust

- Crear workspace de Rust
- Módulos: sensor, edge, coord
- Definir mensajes (JSON)
- Sensor genera datos
- Edge procesa/reenvía
- Coordinator recibe métricas
- Implementar heartbeats

Simulación Red

- Script baseline (limpieza)
- Script de latencia (netem)
- Probar con iperf3 y doc.

Metodología y Gestión del Proyecto

Estrategia Ágil

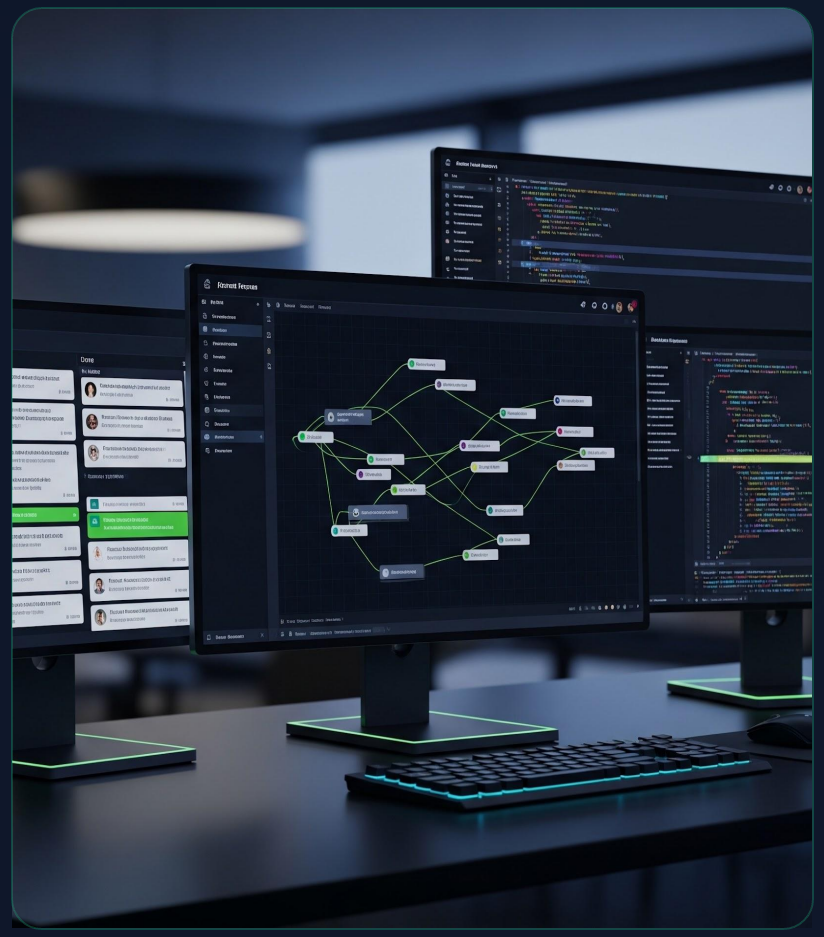
Se implementa una adaptación de Scrum para gestionar la complejidad técnica del monitor de red.

- Sprints: Duración de 2 semana con entregas incrementales.
- Roles: Líder de Proyecto, encargado de Documentación, y Desarrolladores.
- Revisiones: Sincronización diaria para resolver bloqueos.

Herramientas

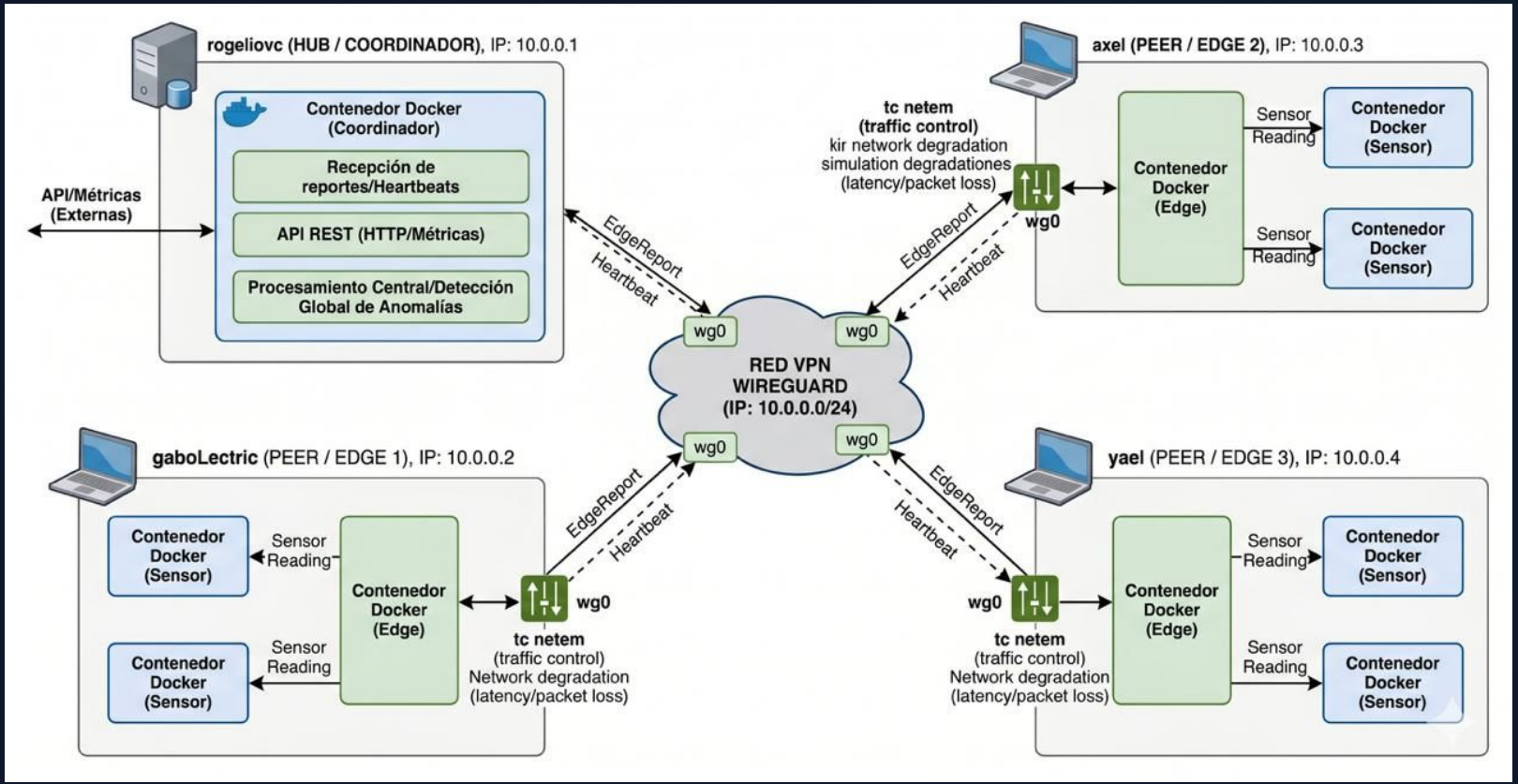
Ecosistema colaborativo para asegurar la trazabilidad y calidad del software.

- Gestión: Github Projects para el seguimiento del Backlog con issues y PRs.
- Código:
- Comunicación: Meet/WhatsApp para validación ágil y técnica.



Topología de Red, VPN, Dockers

Implementación con WireGuard



Nivel de VPN Declarado: [1] (Sin CGNAT, IP pública disponible).

El nodo central (Hub), administrado por Rogelio (rogeliovc), aloja el contenedor del Coordinador y cuenta con una conexión a Internet provista de una dirección IP pública directa, libre de restricciones de Carrier-Grade NAT (CGNAT), por lo que se usó PORT-FORWARDING directamente en el router del domicilio de rogeliovc.

Arquitectura del Pipeline

1. Sensor

Módulo encargado de generar datos simulados. Se ejecuta dentro de su propio contenedor y envía su flujo de información hacia el nodo intermedio (Edge).

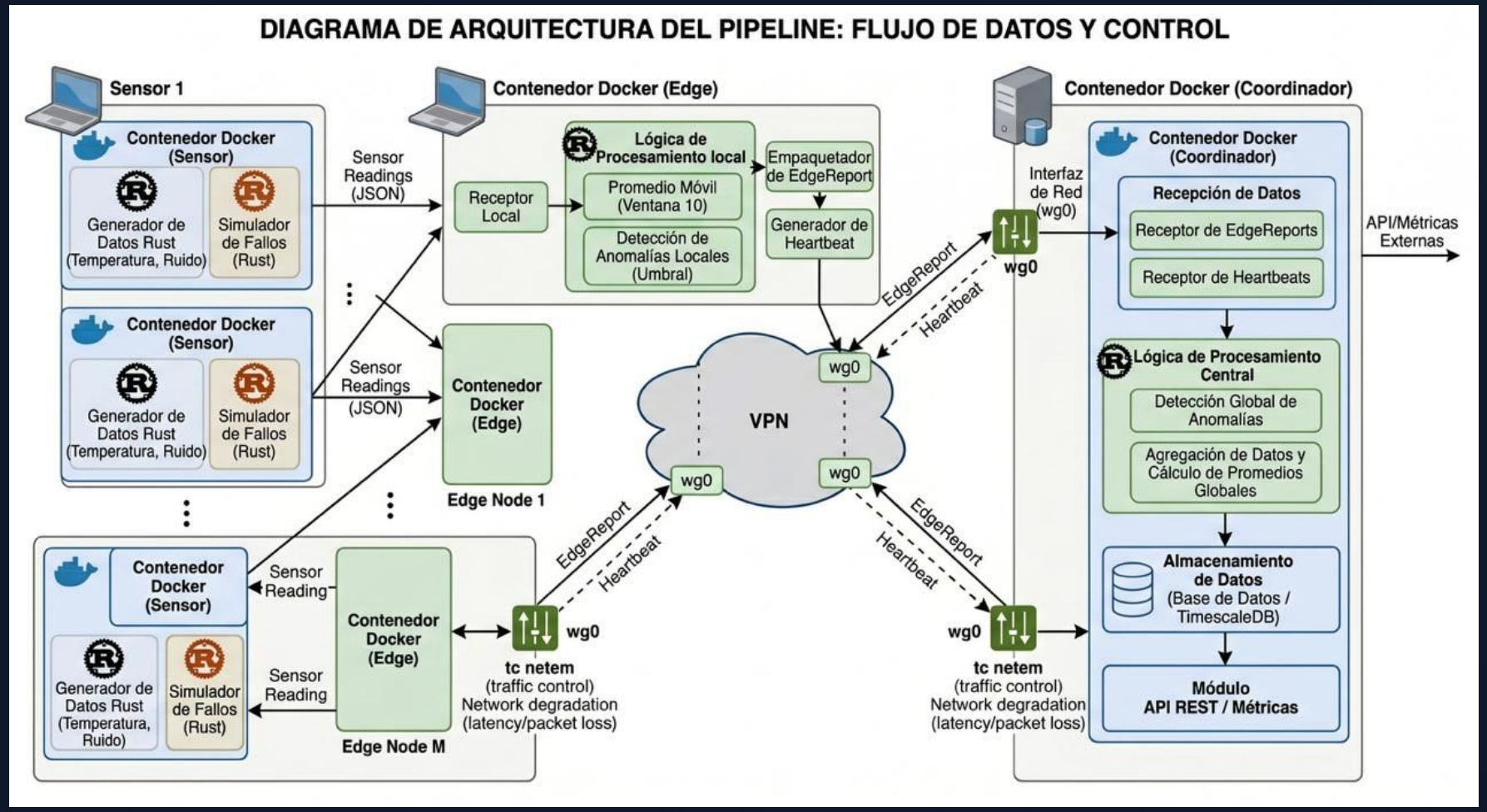
2. Edge Node

Contenedor intermedio crítico. Recibe, procesa y enruta los datos (ej. procesamiento de Mandelbrot). Diseñado con principios de tolerancia a fallos.

3. Coordinator

Nodo central responsable de recolectar y consolidar la información proveniente de múltiples nodos Edge en la red distribuida.

Arquitectura del pipeline



Esta topología asegura que todo el tráfico entre Sensor, Edge y Coordinator viaja encapsulado, permitiendo la inyección de latencia de red realista a nivel de host sin aislar los contenedores Docker.

Prototipo en Rust

El código base ha sido estructurado en espacios de trabajo (workspaces) usando Cargo, promoviendo la modularidad y reutilización:

- **rust/common/:** Definiciones de estructuras de datos compartidas y utilidades de red (`lib.rs`).
- **rust/sensor/:** Binario independiente que actúa como cliente inyector de datos.
- **rust/edge/:** Procesamiento intermedio. Mantiene el estado de conexión para reintentos en caso de fallo.
- **rust/coordinator/:** Servidor principal (`main.rs`) que escucha las conexiones entrantes y centraliza logs.

rust > Cargo.toml

```toml
[workspace]
members = [
    "common",
    "sensor",
    "edge",
    "coordinator"
]
resolver = "2"
```

Prototipo en Rust

Coordinator

main.rs

rust > coordinator > src > main.rs

```rust
use axum::{
    routing::{get, post},
    Json, Router, extract::State,
};
use common::{EdgeReport, Heartbeat, CoordStatus};
use std::collections::HashMap;
use std::sync::{Arc, Mutex};
use std::time::{Duration, SystemTime, UNIX_EPOCH};
use tokio::time;

// Estructura interna para rastrear la salud de cada Edge
#[derive(Debug, Clone)]
struct NodeHealth {
    last_seen_ms: u64,
    is_online: bool,
    expected_seq: u64, // Para rastrear el siguiente mensaje esperado
}

// El estado global del Coordinador
struct AppState {
    edge_nodes: Mutex<HashMap<String, NodeHealth>>,
    total_readings: Mutex<u64>,
    total_anomalies: Mutex<u32>,
    start_time_ms: u64,
    latency_history: Mutex<Vec<u64>>,
    lost_messages: Mutex<u64>,
}

#[tokio::main]
async fn main() {
    let port = std::env::var("PORT").unwrap_or_else(|_| "3000".to_string());

    let state = Arc::new(AppState {
        edge_nodes: Mutex::new(HashMap::new()),
        total_readings: Mutex::new(0),
        total_anomalies: Mutex::new(0),
        start_time_ms: current_time_ms(),
        latency_history: Mutex::new(Vec::with_capacity(1000)), // Evita realojamientos
        lost_messages: Mutex::new(0),
    });

    let bg_state = state.clone();

```

Edge

main.rs

rust > edge > src > main.rs

```rust
use axum::{
    routing::post,
    Json, Router, extract::State,
};
use common::{SensorReading, EdgeReport};
use std::sync::{Arc, Mutex};
use std::time::{SystemTime, UNIX_EPOCH, Duration};
use tokio::time;
use std::sync::atomic::{AtomicU64, Ordering};

// Este es el estado compartido en memoria.
// Axum guarda las lecturas aquí, y la tarea en segundo plano las toma de aquí.
struct AppState {
    readings: Arc<Mutex<Vec<SensorReading>>>,
    edge_id: String,
    coordinator_url: String,
    coord_hb_url: String,
    sequence_counter: Arc<AtomicU64>,
}

#[tokio::main]
async fn main() {
    // Configuración con variables de entorno para Docker
    let edge_id = std::env::var("EDGE_ID").expect("FALTA VARIABLE: EDGE_ID");
    let coordinator_url = std::env::var("COORDINATOR_URL").expect("FALTA VARIABLE: COORDINATOR_URL");
    let coord_hb_url = std::env::var("COORD_HB_URL").expect("FALTA VARIABLE: COORD_HB_URL");
    let port = std::env::var("PORT").unwrap_or_else(|_| "4000".to_string());

    // Inicializamos el estado vacío
    let shared_state = Arc::new(AppState {
        readings: Arc::new(Mutex::new(Vec::new())),
        edge_id: edge_id.clone(),
        coordinator_url: coordinator_url.clone(),
        coord_hb_url: coord_hb_url.clone(),
        sequence_counter: Arc::new(AtomicU64::new(0)),
    });

    // Clonamos el puntero del estado para dárselo al hilo en segundo plano
    let bg_state = shared_state.clone();

```

Prototipo en Rust

Sensor

main.rs

rust > sensor > src > main.rs

```rust
use common::SensorReading;
use std::time::{SystemTime, UNIX_EPOCH, Duration};
use rand::Rng;
use tokio::time;

#[tokio::main]
async fn main() {
    // Usamos variables de entorno para que Docker Compose pueda inyectar las IPs reales
    let sensor_id = std::env::var("SENSOR_ID").expect("FALTA VARIABLE: SENSOR_ID");
    let edge_url = std::env::var("EDGE_URL").expect("FALTA VARIABLE: EDGE_URL");
    let coord_hb_url = std::env::var("COORD_HB_URL").expect("FALTA VARIABLE: COORD_HB_URL");

    println!("🚀 Iniciando {}...", sensor_id);
    println!("📡 Enviando ráfagas de datos a {}", edge_url);
    println!("❤️ Heartbeat configurado hacia {}", coord_hb_url);

    let client = reqwest::Client::new();

    // Hilo de Heartbeat para el Sensor
    let sensor_id_hb = sensor_id.clone();
    tokio::spawn(async move {
        let client = reqwest::Client::new();
        let mut interval = time::interval(Duration::from_secs(2));
        loop {
            interval.tick().await;
            let hb = common::Heartbeat {
                node_id: sensor_id_hb.clone(),
                role: "sensor".to_string(),
                timestamp_ms: SystemTime::now().duration_since(UNIX_EPOCH).unwrap().as_millis() as u64,
            };
            let _ = client.post(&coord_hb_url).json(&hb).send().await;
        }
    });

    // Configuramos el sensor para enviar 2 lecturas por segundo (cada 500ms)
    let mut interval = time::interval(Duration::from_millis(500));
    let mut rng = rand::thread_rng();

    loop {
        interval.tick().await;

        // Temperatura base aleatoria (ej. 20.0 a 30.0 grados)
        let base_temp: f64 = rng.gen_range(20.0..30.0);

        // Simular una anomalía térmica (> 80.0) con un 5% de probabilidad
        let temp = if rng.gen_bool(0.05) {
            rng.gen_range(85.0..95.0)
        } else {
            base_temp
        };
```

Elementos de la Demostración

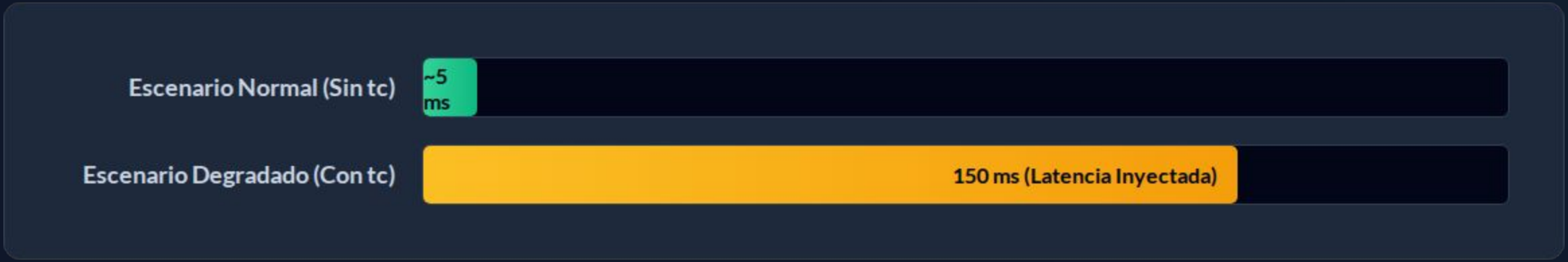
Estado de la VPN Comprobación en vivo del estado del túnel usando comandos como wg show para confirmar el handshake.

Estado de Contenedores Ejecución de docker ps para verificar la salud y presencia de los tres tipos de contenedores orquestados.

Flujo en Tiempo Real Visualización de logs del *coordinator* y *edge nodes* para demostrar la correcta recepción y ruteo de datos.

Escenario con tc netem Aplicación en vivo de un escenario de degradación de red para demostrar el impacto en el pipeline.

Pruebas de Red con tc netem



La demostración validará mediante `iperf3` (o logs de procesamiento) el comportamiento del pipeline en la recepción de datos bajo condiciones de red adversas controladas.

Pruebas de Red sin tc netem

```
coordinator  | [HEARTBEAT] Recibido de edge-axel - Latencia: 296ms
coordinator  | Reporte normal procesado desde edge-axel (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-gaboLectric - Latencia: 110ms
coordinator  | Reporte normal procesado desde edge-gaboLectric (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-yael - Latencia: 0ms
coordinator  | Reporte normal procesado desde edge-yael (10 muestras)
coordinator  | Reporte normal procesado desde edge-rogelio (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-rogelio - Latencia: 1ms
coordinator  | [HEARTBEAT] Recibido de edge-axel - Latencia: 264ms
coordinator  | [HEARTBEAT] Recibido de edge-gaboLectric - Latencia: 80ms
coordinator  | [HEARTBEAT] Recibido de edge-yael - Latencia: 13ms
coordinator  | Reporte normal procesado desde edge-yael (10 muestras)
coordinator  | Reporte normal procesado desde edge-rogelio (10 muestras)
coordinator  | Reporte normal procesado desde edge-axel (10 muestras)
coordinator  | Reporte normal procesado desde edge-gaboLectric (10 muestras)
coordinator  | Reporte normal procesado desde edge-yael (10 muestras)
coordinator  | Reporte normal procesado desde edge-axel (10 muestras)
coordinator  | Reporte normal procesado desde edge-rogelio (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-rogelio - Latencia: 1ms
coordinator  | Reporte normal procesado desde edge-gaboLectric (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-gaboLectric - Latencia: 53ms
coordinator  | [HEARTBEAT] Recibido de edge-yael - Latencia: 54ms
coordinator  | [HEARTBEAT] Recibido de edge-axel - Latencia: 675ms
coordinator  | Reporte normal procesado desde edge-yael (10 muestras)
coordinator  | Reporte normal procesado desde edge-rogelio (10 muestras)
coordinator  | Reporte normal procesado desde edge-axel (10 muestras)
coordinator  | Reporte normal procesado desde edge-gaboLectric (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-rogelio - Latencia: 1ms
coordinator  | [HEARTBEAT] Recibido de edge-axel - Latencia: 309ms
coordinator  | [HEARTBEAT] Recibido de edge-gaboLectric - Latencia: 124ms
coordinator  | [HEARTBEAT] Recibido de edge-yael - Latencia: 25ms
coordinator  | Reporte normal procesado desde edge-axel (10 muestras)
coordinator  | Reporte normal procesado desde edge-gaboLectric (10 muestras)
coordinator  | Reporte normal procesado desde edge-yael (10 muestras)
```

Escenario base sin degradación artificial para contrastar la latencia y el rendimiento nominal del sistema.

Pruebas de Red con tc netem

```
coordinator  | Reporte normal procesado desde edge-yael (10 muestras)
coordinator  | Reporte normal procesado desde edge-rogelio (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-gaboLectric - Latencia: 2012ms
coordinator  | [HEARTBEAT] Recibido de edge-rogelio - Latencia: 1ms
coordinator  | Reporte normal procesado desde edge-gaboLectric (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-yael - Latencia: 25ms
coordinator  | Reporte normal procesado desde edge-gaboLectric (10 muestras)
coordinator  | Reporte normal procesado desde edge-yael (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-gaboLectric - Latencia: 2609ms
coordinator  | Reporte normal procesado desde edge-rogelio (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-rogelio - Latencia: 1ms
coordinator  | [HEARTBEAT] Recibido de edge-yael - Latencia: 0ms
coordinator  | Reporte normal procesado desde edge-gaboLectric (10 muestras)
coordinator  | Reporte normal procesado desde edge-yael (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-gaboLectric - Latencia: 1749ms
coordinator  | Reporte normal procesado desde edge-rogelio (10 muestras)
coordinator  | Reporte normal procesado desde edge-gaboLectric (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-rogelio - Latencia: 0ms
coordinator  | [HEARTBEAT] Recibido de edge-yael - Latencia: 0ms
coordinator  | Reporte normal procesado desde edge-yael (10 muestras)
coordinator  | Reporte normal procesado desde edge-rogelio (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-gaboLectric - Latencia: 1840ms
coordinator  | Reporte normal procesado desde edge-yael (10 muestras)
coordinator  | Reporte normal procesado desde edge-rogelio (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-rogelio - Latencia: 0ms
coordinator  | [HEARTBEAT] Recibido de edge-yael - Latencia: 0ms
coordinator  | Reporte normal procesado desde edge-gaboLectric (10 muestras)
coordinator  | Reporte normal procesado desde edge-gaboLectric (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-gaboLectric - Latencia: 2611ms
coordinator  | [HEARTBEAT] Recibido de edge-rogelio - Latencia: 1ms
coordinator  | [HEARTBEAT] Recibido de edge-yael - Latencia: 0ms
coordinator  | Reporte normal procesado desde edge-yael (10 muestras)
coordinator  | Reporte normal procesado desde edge-rogelio (10 muestras)
coordinator  | Reporte normal procesado desde edge-gaboLectric (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-gaboLectric - Latencia: 2068ms
coordinator  | Reporte normal procesado desde edge-yael (10 muestras)
coordinator  | Reporte normal procesado desde edge-rogelio (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-rogelio - Latencia: 0ms
coordinator  | [HEARTBEAT] Recibido de edge-yael - Latencia: 87ms
coordinator  | Reporte normal procesado desde edge-gaboLectric (10 muestras)
coordinator  | Reporte normal procesado desde edge-yael (10 muestras)
coordinator  | Reporte normal procesado desde edge-rogelio (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-gaboLectric - Latencia: 2152ms
coordinator  | [HEARTBEAT] Recibido de edge-rogelio - Latencia: 0ms
```

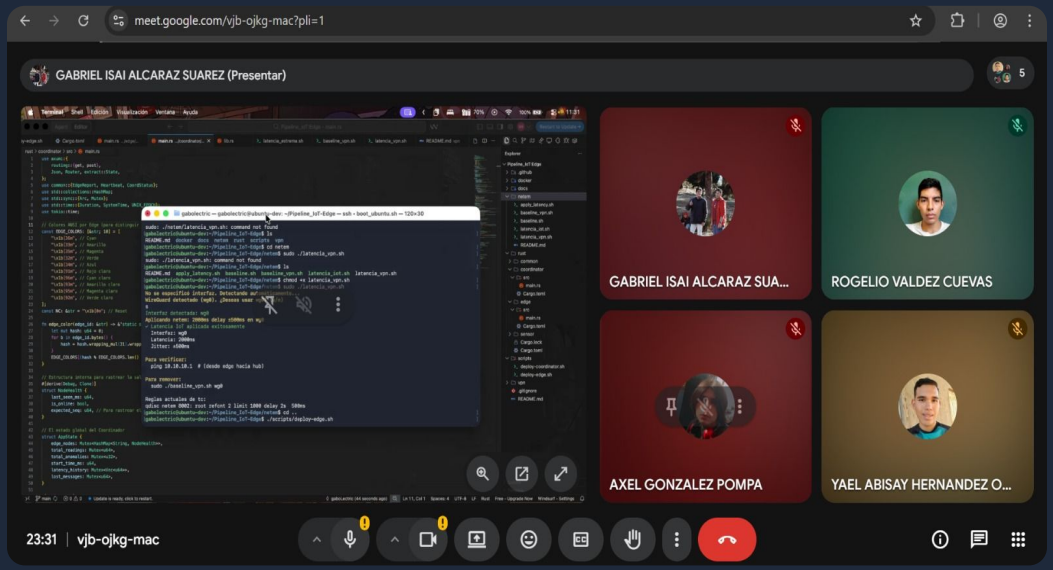
Impacto de tc netem

A diferencia del escenario base, la inyección de reglas mediante **tc netem** introduce una degradación artificial controlada:

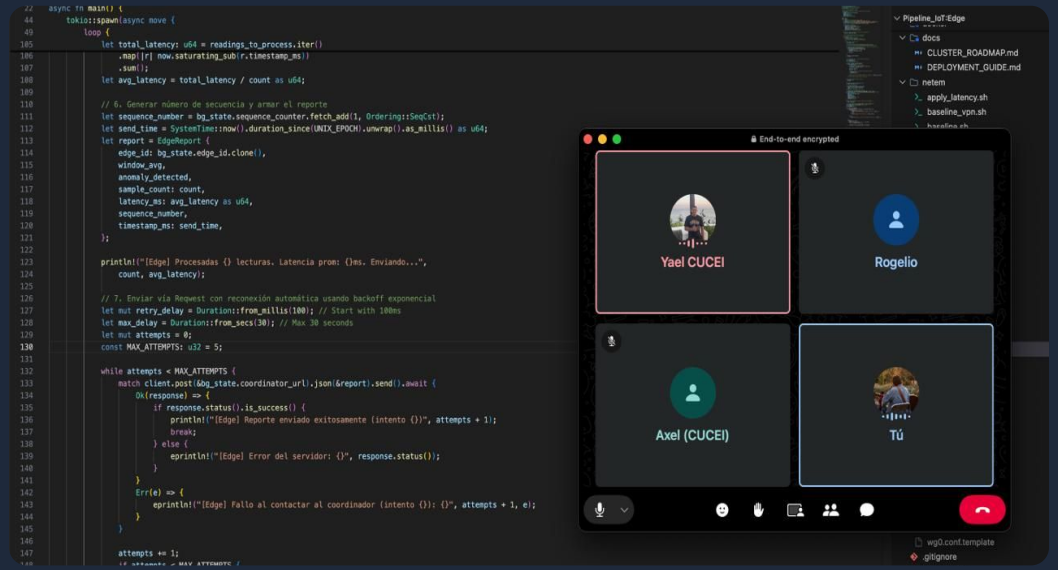
- **Aumento de Latencia:** Se observa un incremento significativo en el tiempo de respuesta.
- **Variabilidad (Jitter):** La latencia no es constante, afectando la regularidad del flujo de datos del sensor.
- **Efecto en Pipeline:** El sistema debe gestionar buffers y tiempos de espera (timeouts) más largos para evitar la pérdida de mensajes.

.

Evidencia de Reuniones y Validación Ágil



Reunión Sprint 1 (Sincronización): Confirmar la arquitectura del sistema de tres nodos lógicos



Reunión Sprint 1 (Técnica): Definición de reglas tc netem y arquitectura del monitor de red.Validación de Edge y Configuración de Red

Estas evidencias validan el trabajo colaborativo requerido para el Proyecto 2, asegurando la trazabilidad de las decisiones técnicas y la asignación equitativa de tareas.

Retos Encontrados

Sincronización Docker / VPN

Asegurar que los contenedores enruten correctamente el tráfico interno a través de la interfaz wg0 del host sin comprometer su aislamiento de red predeterminado.

Flujo Asíncrono en Rust

Implementación concurrente utilizando el framework Tokio para evitar el bloqueo (race conditions) durante la recepción y reenvío continuo de telemetría.

Aislamiento de Reglas tc

Aplicar correctamente Traffic Control (tc netem) de modo que afecte el tráfico de extremo a extremo, pero se mida correctamente antes de la desencapsulación.

¿Preguntas?

Paso a la demostración técnica en vivo en la terminal.